AEGIS-AGENT

Antigravity prompt · Front end only

```
Build the FRONT END ONLY for a hackathon prototype called AEGIS-AGENT.

PROJECT CONTEXT
AEGIS-AGENT is a multi-agent security workflow for an authorized, local test application. The
   future system will have four roles:
1. Red Agent: runs a selected, predefined security test.
2. Sentinel Agent: reviews the result and evidence.
3. Fix Agent: proposes a remediation.
4. Auditor Agent: displays regression test results.

For this task, create only the user interface and front-end interactions. Do not build a
   backend, connect to real agents, scan targets, run exploits, modify source code, create
   pull requests, or call external APIs. Use realistic mock data and simulated progress so the
   interface can be demonstrated on its own.

GOAL
Create a polished, responsive “security war room” dashboard that clearly shows one run moving
   through:
Scope accepted → Test run → Evidence reviewed → Fix proposed → Regression checked

PAGES / AREAS
1. Dashboard
   - Project name and short description.
   - A prominent “Start Demo Run” button.
   - A recent runs list with status, target label, finding count, and time.
   - A short notice that the prototype is for authorized local testing only.

2. New Run setup
   - Target field prefilled with “Local training app”.
   - Test case selector with a few safe demo options, such as “Input validation check” and
      “Authentication check”.
   - A clear scope confirmation checkbox.
   - “Start run” and “Cancel” actions.
   - Do not allow arbitrary public URLs; this is a UI prototype and must not imply that real
      scanning occurs.

3. Run details / live timeline
   - Show the five workflow stages as a readable timeline with states: queued, running,
      complete, needs review, or failed.
   - Simulate progress when a user starts a demo run. Add short delays between stages and
      update the UI visibly.
   - Show each agent’s role, status, concise activity message, and timestamp.
   - Include a “Pause demo” or “Reset demo” control.

4. Finding and evidence view
   - Show one clearly labeled mock finding.
   - Include severity, affected component, a short explanation, and sample evidence.
   - Mark all information as simulated demo data.
   - Provide “Confirmed”, “Not confirmed”, and “Needs review” as visible review states, but
      make clear these are prototype UI controls.

5. Proposed fix view
   - Show a mock remediation summary and a readable sample diff or before/after snippet.
   - Include “Approve for demo” and “Reject” controls that only update the front-end state.
   - Do not claim that code was actually changed or a pull request was created.

6. Regression result
   - Display mocked checks with pass/fail status.
   - Show whether the original demo test is still reproducible.
   - Distinguish simulated results from real test execution.
```

DESIGN
- Professional dark security-operations interface, with a restrained palette, high contrast, and clear status colors.
- Make the workflow and evidence the visual focus. Avoid a cluttered grid of tiny cards.
- Use accessible labels, keyboard-friendly controls, and responsive layouts for desktop and mobile.
- Use plain, direct language. Avoid unsupported claims like “zero false positives”, “zero-day discovery”, or guaranteed remediation.
- Include a small persistent label: “Front-end demo — no real scans or code changes”.

IMPLEMENTATION
- First inspect the existing project and follow its framework and conventions.
- If no app exists, create a simple, maintainable front end using the environment’s existing tools.
- Keep mock data and demo state organized so a backend can be connected later.
- Include sensible empty, loading, success, failure, and needs-review states.
- Do not add backend services, credentials, external integrations, or real security-testing functionality.

DELIVERABLE
Leave the app in a runnable state and provide the command to start it, plus a brief list of the screens and interactions implemented.